Omni-Attention: A Unified High-Performance Compute Framework for Heterogeneous AI Infrastructure

1 Executive Summary: Architecting for Scale

The contemporary landscape of Artificial Intelligence infrastructure is defined by a singular, relentless economic pressure: the cost of compute. As foundation models scale beyond the trillion-parameter mark, building and optimizing infrastructure has evolved from a support function into a critical strategic asset. The differentiation between standard development and state-of-the-art engineering lies in the transition from consumer to architect.

Most engineers consume libraries; they import PyTorch, configure a DataLoader, and rely on the compiler (XLA or Inductor) to handle the complexities of execution. The systems architect, however, builds the primitives that enable those libraries to function at scale. They possess a rare, poly-accelerator capability: the ability to reason about memory hierarchies, compiler Intermediate Representations (IR), and distributed topologies across fundamentally different hardware architectures.

This report proposes and details the execution of "Omni-Attention," a comprehensive engineering project designed to demonstrate this specific mastery. The project involves building a unified, distributed attention mechanism that operates natively on both NVIDIA Hopper GPUs (via OpenAI Triton) and Google TPU v5e/Trillium architectures (via JAX Pallas). Crucially, to satisfy the rigorous infrastructure requirements of modern cloud deployments, the project includes "Omni-Serve," a custom C++ inference runtime that orchestrates these kernels without the overhead of Python.

This document serves as an exhaustive technical blueprint. It dissects the hardware physics of the H100 and TPU v5e, analyzes the software abstractions of Triton and Pallas, and details the C++ PJRT integration required to validate this architecture in a production-grade systems environment.

2 The Architectural Divergence: Hopper vs. TPU v5e

To engineer at the cutting edge, one must first understand the silicon. Modern AI accelerators target two distinct architectural philosophies: the Many-Core SIMT (Single Instruction, Multiple Threads) approach of NVIDIA and the VLIW (Very Long Instruction Word) Systolic Array approach of Google. A unified framework must demonstrate fluency across both.

2.1 NVIDIA H100 (Hopper): The Asynchronous Dataflow Engine

The NVIDIA H100 represents a paradigm shift from its predecessor, the A100. For GPU optimization, the critical evolution is the move toward asynchrony. In previous architectures (Volta/Ampere), threads were responsible for issuing load instructions, waiting for data, and then computing. This synchronous model left the massive math units idle while waiting for memory.

2.1.1 Tensor Memory Accelerator (TMA)

The H100 introduces the Tensor Memory Accelerator (TMA), a dedicated hardware block that decouples memory movement from thread execution.

- **Mechanism:** The TMA allows a generic CUDA warp (or Triton program) to issue a "copy descriptor" to the memory subsystem. The TMA then asynchronously fetches multi-dimensional tensor blocks from High Bandwidth Memory (HBM) directly into Shared Memory (SRAM), handling all address calculations and boundary predicates in hardware.
- **Implication for Performance:** This frees the Streaming Multiprocessors (SMs) to perform math (WGMMA) or other operations. The programmer's task shifts from "optimizing loads" to "orchestrating pipelines," ensuring that the TMA is always busy fetching the next tile while the Tensor Cores consume the current tile. This is the foundation of FlashAttention-3.

2.1.2 WGMMA (Warpgroup Matrix Multiply-Accumulate)

Hopper replaces the older MMA instructions with WGMMA.

- **Mechanism:** WGMMA allows a "warpgroup" (four contiguous warps, 128 threads) to collaborate on a dense matrix multiplication.
- **Architectural Insight:** This increases the "arithmetic intensity" of the instruction stream. A single instruction triggers a massive amount of compute, reducing the pressure on the instruction cache and the warp scheduler. It forces the programmer to think in terms of 128-thread cooperative blocks rather than individual thread behavior.

2.2 Google TPU v5e: The Systolic Dataflow Engine

On Google's Cloud AI infrastructure, the paradigm is radically different. The TPU is not a collection of threads; it is a massive systolic array designed to pump matrices through processing units deterministically.

2.2.1 The Matrix Multiply Unit (MXU) and Determinism

The heart of the TPU is the MXU, a dense grid (e.g., 128×128) of bfloat16 multipliers. Data flows in from the left and top, and partial sums flow down.

- **The Constraint:** This architecture is rigid. It excels at dense matrix multiplication but struggles with irregular memory access patterns (scatter/gather) or operations that break the systolic flow.
- **The VPU Interaction:** Non-matmul operations (Softmax, LayerNorm, RoPE) are handled by the Vector Processing Unit (VPU). If the compiler (XLA) cannot fuse these effectively, or if the layout requires transposing data between the MXU and VPU, performance collapses. This is why "out of the box" XLA performance sometimes plateaus and why custom kernels (Pallas) are necessary for optimal efficiency.

2.2.2 Explicit Memory Hierarchy: VMEM vs. SMEM

Unlike the GPU's managed cache hierarchy, the TPU exposes its memory physically.

- **VMEM (Vector Memory):** A fast, on-chip scratchpad (tens of megabytes) that feeds the arithmetic units.
- **SMEM (Scalar Memory):** A small memory space for control flow scalars and addresses.
- **The Programmer's Task:** Programming a TPU involves explicitly managing the Direct Memory Access (DMA) engine to stage data from HBM to VMEM. The engineer is the cache controller. This level of control is what Pallas enables, distinguishing it from high-level JAX.

Feature	NVIDIA H100 (GPU)	Google TPU v5e (TPU)
Compute Core	Streaming Multiprocessor (SM)	Matrix Multiply Unit (MXU) + VPU
Parallelism Model	SIMT (Threads/Warps)	SIMD / Systolic Dataflow
Memory Management	HBM → L2 → Shared Mem (Hardware Cache + TMA)	HBM → VMEM (Explicit DMA)
Programming Model	CUDA/PTX/Triton	XLA HLO / Mosaic / Pallas
Interconnect	NVLink/NVSwitch	ICI (Inter-Chip Interconnect) / Torus

Table 1: Architectural comparison highlighting the divergent technical details of the target architectures.

2.3 Synthesis: The "Poly-Accelerator" Competency

The "Omni-Attention" project bridges this gap. By implementing the same mathematical operation (Attention) on both architectures using their respective low-level languages (Triton and Pallas), it demonstrates a foundational engineering "meta-skill": the ability to reason about compute abstractly, applying the correct tiling and pipelining strategies regardless of the underlying silicon.

3 The GPU Frontier: Next-Generation Kernels with OpenAI Triton

The first pillar of the project targets the requirements for state-of-the-art GPU performance. The objective is to build a FlashAttention-3 compatible kernel using OpenAI Triton. Standard FlashAttention-2 is no longer the ceiling; efficient utilization of Hopper's asynchronous features is the new standard.

3.1 The Triton Programming Model

Triton is an intermediate language that compiles to PTX (Parallel Thread Execution). It allows Python-like syntax to express block-level operations.

- **Block Pointers (`tl.make_block_ptr`):** To utilize the H100's TMA, one cannot use simple pointer arithmetic. One must define a "block pointer"—a 2D descriptor of the tensor in memory. This abstraction maps directly to the TMA hardware descriptor, allowing the compiler to emit `cp.async.bulk.tensor` instructions.
- **Insight:** An advanced implementation uses block pointers not just for convenience, but to enforce memory coalescing and enable the hardware prefetcher.

3.2 Implementing FlashAttention-3 Logic

FlashAttention-3 introduces "Warp Specialization" to overlap the Softmax operation with the GEMM (General Matrix Multiply) operation. On H100, the Tensor Cores (GEMM) and the ALU (Softmax/-Exp) are separate pipelines. If we run them sequentially, we leave performance on the table.

3.2.1 Asynchronous Pipeline Design

The kernel must be structured to maximize overlap.

- **Stage 1: Producer (TMA):** Issue asynchronous loads for Block K_{i+1} and V_{i+1} while processing Block K_i and V_i.

- **Stage 2: Consumer (GEMM/Softmax):**
  - Compute $S = Q \cdot K_i^T$ (GEMM).
  - Simultaneously (if possible via warpgroups) or interleaved: Compute $P = \text{softmax}(S)$.
  - Compute $O = P \cdot V_i$ (GEMM).

Code Conceptualization (Triton):

```
# Conceptual FlashAttention-3 Pipeline in Triton
@triton.jit
def flash_attn_fwd_kernel(Q, K, V,...):
    # Define Block Pointers for TMA
    K_block_ptr = tl.make_block_ptr(base=K, shape=(DIV, SEQ_LEN),
                                    strides..., block_shape=(BLOCK_D, BLOCK_N))
    V_block_ptr = tl.make_block_ptr(base=V, shape=(SEQ_LEN, DIV),
                                    strides..., block_shape=(BLOCK_N, BLOCK_D))

    # Prologue: Load first block
    k_tile = tl.load(K_block_ptr)
    v_tile = tl.load(V_block_ptr)

    for i in range(LOOP_ITER):
        # 1. Issue Async Load for NEXT block
        tl.advance(K_block_ptr, (0, BLOCK_N))
        tl.advance(V_block_ptr, (BLOCK_N, 0))
        k_next = tl.load(K_block_ptr) # Async Load instruction
        v_next = tl.load(V_block_ptr)

        # 2. Compute Current GEMM (Q @ K)
        # WGMMA instruction emitted here
        qk = tl.dot(q_tile, k_tile)

        # 3. Compute Softmax (ALU)
        # Compiler optimization: Interleave ALU ops with WGMMA Latency
        m = tl.max(qk, 1)
        p = tl.exp(qk - m)

        # 4. Compute Output GEMM (P @ V)
        acc += tl.dot(p, v_tile)

        # 5. Pipeline Advance
        k_tile = k_next
        v_tile = v_next
```

Analysis: The explicit loading of `_next` before processing `_tile` is the software pipelining pattern. On H100, this enables the TMA to fetch data from global memory into shared memory while the Tensor Cores are crunching the dot product of the previous tile. This hides the latency of the HBM access, addressing the "Memory Wall".

3.3 Layout Optimization: Swizzling to Avoid L2 Thrashing

A subtle but critical optimization required for peak performance is understanding the L2 cache.

- **The Problem:** If multiple thread blocks (CTAs) read from the same DRAM rows simultaneously, they can cause "partition camping" or cache thrashing, reducing effective bandwidth.
- **The Solution (Swizzling):** Instead of assigning blocks in a naive row-major order (Block 0, Block 1, Block 2...), we use a space-filling curve (like a Hilbert curve) or a simple swizzle logic:

```
pid_swizzled = (pid_m ^ pid_n)
```

- **Effect:** This ensures that adjacent thread blocks work on spatially local data in the grid, increasing the "hit rate" of the L2 cache for the shared K and V blocks.

4 The TPU Frontier: Custom Operations with JAX Pallas

The second pillar targets Google's Cloud AI infrastructure. While XLA is a powerful compiler, it is a heuristic engine. It makes "safe" decisions that are often suboptimal for bleeding-edge research architectures. Pallas provides the "escape hatch" necessary to implement logic that XLA refuses to generate.

4.1 The Pallas Programming Model

Pallas allows the user to write kernels using JAX syntax but with explicit control over memory and scheduling. It maps to the Mosaic backend on TPU.

- **The Grid:** Similar to a CUDA grid, Pallas kernels launch over a domain.
- **The Ref:** Unlike JAX arrays (immutable), Pallas operates on `Ref` objects (mutable pointers to VMEM). This allows for in-place accumulation, which is essential for FlashAttention.

4.2 Explicit Memory Management: The BlockSpec

The most distinctive feature of TPU programming in Pallas is the `BlockSpec`. This object defines the DMA schedule.

- **Mechanism:** When a Pallas kernel is launched, the `BlockSpec` tells the Direct Memory Access (DMA) engine: "Before the kernel starts on the core, copy slice $\Phi[x : x + 128, :]$ from HBM to VMEM."
- **Pipelining:** Pallas supports explicit pipelining via `pl.pipeline`. This creates a multi-stage pipeline where the DMA engine fetches stage $N + 1$ into a "scratch" buffer in VMEM while the MXU processes stage N.
- **Relevance:** Tuning the `num_stages` and `scratch_shape` parameters in Pallas is exactly the kind of "hardware-aware" optimization required for peak TPU utilization.

4.3 Handling "Megacore" and Vector Register Pressure

TPU v4/v5 architectures use a "Megacore" design where two independent cores share a VPU.

- **Challenge:** If both cores try to execute heavy transcendental operations (Softmax) simultaneously, they contend for the VPU, stalling the MXU.
- **Pallas Solution:** A custom kernel allows the engineer to "dimension" the workload such that the VPU usage is minimized or interleaved. Furthermore, the workload must be tuned via tile size (e.g., 128×128) to fit within the vector register file. If the tile is too large, the compiler "spills" to VMEM, causing a massive performance regression. The project includes a "tuning sweep" script that empirically finds the optimal block size for the v5e architecture.

5 The Distributed Frontier: Ring Attention and `shard_map`

A kernel that runs fast on one chip is insufficient. Modern models (> 100 billion parameters) do not fit on a single chip. Large-scale infrastructure demands expertise in Distributed Training. The Omni-Attention project addresses this via Ring Attention.

5.1 The Logic of Ring Attention

As sequence length (S) scales, the memory required for the Key/Value cache ($2 \cdot S \cdot H$) exceeds the HBM of a single device. Ring Attention distributes the sequence across N devices.

Algorithm:

1. Device i holds Q_i, K_i, V_i (a slice of the full sequence).
2. Device i computes attention between Q_i and local K_i, V_i.
3. **Rotate:** Device i sends K_i, V_i to Device $i+1$ and receives K_{i-1}, V_{i-1}.
4. Device i computes attention between Q_i and the received K, V.
5. Repeat $N-1$ times.

5.2 Implementing with `shard_map`

`shard_map` (SPMD - Single Program Multiple Data) is the modern JAX primitive for distributed control. It replaces the older `pmap` and `xmap`.

5.2.1 Defining the Mesh

We define a physical mesh of devices.

```
mesh = jax.sharding.Mesh(devices, ('data', 'model', 'ring'))
```

The "ring" axis corresponds to the dimension along which the sequence is sharded.

5.2.2 The Collective: `lax.ppermute`

Inside the `shard_map` function, we use `lax.ppermute` to implement the rotation.

- **Topology Awareness:** On TPU, the interconnect (ICI) is a 3D torus. `lax.ppermute` maps efficiently to neighbor-to-neighbor shifts on this torus. An optimal implementation maps the "ring" axis of the logical mesh to the correct physical axis of the TPU torus (e.g., the dimension with the highest bandwidth) as a critical optimization.

5.2.3 Overlap and Latency Hiding

The holy grail of distributed training is Computation-Communication Overlap.

- **The Strategy:** While the `ppermute` is transferring the K, V blocks for the next iteration, the device must be computing the attention for the current iteration.
- **Implementation:** In JAX, this is often handled by the compiler, but `shard_map` allows for explicit expression of independence. By structuring the loop such that the `ppermute` output is not needed until the end of the computation block, we allow XLA to schedule the DMA transfer in parallel with the MXU operations.
- **Verification:** The report details the use of the Trace Viewer to confirm this overlap. A successful implementation shows the "Collective Permute" bar running in parallel with the "Fusion" (Compute) bar.

6 Precision Engineering: The FP8 Paradigm Shift

Both the NVIDIA H100 and TPU v5e are optimized for 8-bit floating point (FP8) math. Implementing this is a mandatory requirement for a "modern" infrastructure project.

6.1 The Two Flavors of FP8

FP8 is not a single format; it is a duality designed to balance dynamic range and precision.

1. **E4M3 (4 exponent, 3 mantissa):**
   - **Characteristics:** Higher precision, lower dynamic range.
   - **Use Case:** Forward pass activations and weights. The distributions of weights are typically Gaussian and well-behaved, fitting within the limited range.
2. **E5M2 (5 exponent, 2 mantissa):**
   - **Characteristics:** Lower precision, higher dynamic range (matches FP16 range).
   - **Use Case:** Gradients. Gradients in the backward pass can have large outliers ("spikes"). E4M3 would overflow (yielding NaN), causing training divergence. E5M2 handles these outliers.

6.2 Block-Wise Quantization (Microscaling)

A single scale factor for an entire tensor is insufficient for LLMs due to "outlier features."

- **The Technique:** We divide the tensor into blocks (e.g., 128×128). We calculate a separate scale factor for each block.

$$Scale_{\text{block}} = \frac{Max_{FP8}}{Max(|Tensor_{\text{block}}|)}$$

- **Hardware Implementation:**
  - **Triton:** The project implements a fused kernel that loads FP32/BF16 data, computes the max of the block, scales it, casts to FP8, and stores both the FP8 data and the scale factors.
  - **Challenge:** The H100 WGMMA instructions for FP8 require specific layouts. The Triton kernel must ensure the data is swizzled correctly during the quantization step to avoid penalties during the consumption step.

6.3 Numerics and Stability

Training in FP8 is risky. A robust framework must ensure convergence.

- **Stochastic Rounding:** Simply truncating to FP8 introduces bias. The project implementation should consider stochastic rounding (adding random noise before truncation) to preserve the statistical properties of the gradients.
- **Unit Tests:** The project report must define a testing strategy. Comparing the FP8 kernel output against an FP32 baseline and measuring the Root Mean Square Error (RMSE). The goal is not zero error, but error within the tolerance of the model's noise floor.

7 Profiling and Verification: The "Detective" Work

The section on profiling transforms the project from a theoretical implementation into a validated engineering artifact.

7.1 GPU Profiling with Nsight Compute (NCU)

The report details the use of `ncu` to analyze the Triton kernel.

- **The Roofline Model:** We plot Operational Intensity (FLOPs/Byte) vs. Performance (TFLOPS).
  - **Memory Bound:** If the kernel is under the slanted part of the roofline, we are waiting for HBM. Fix: Improve pipelining (TMA), increase tile size to reuse data.
  - **Compute Bound:** If we are under the flat part, we are limited by the Tensor Cores. Fix: Ensure WGMMA instructions are being issued every cycle, check for "Stalls" (e.g., Math Pipe Throttle).
- **Bank Conflicts:** `ncu` will report "Shared Memory Bank Conflicts." The project explicitly addresses this via the swizzling layout mentioned in Section 3.3.

7.2 TPU Profiling with JAX Profiler

For the TPU backend, we use the JAX Profiler.

- **Step Trace:** We analyze the "Step Time."
- **HBM Bandwidth:** The "Memory Viewer" shows the utilization of the HBM bus. A Pallas kernel failing to saturate bandwidth during load phases indicates poor `BlockSpec` tuning (e.g., single buffering instead of double buffering).
- **Idle Time:** The "Trace Viewer" visualizes gaps. Red "formatting" ops (like padding or transposes) inserted by XLA around the Pallas kernel are the enemy. The report discusses strategies to align the Pallas kernel's expected layout with the JAX array layout to eliminate these copies.

7.3 The Metric: Model Flops Utilization (MFU)

The ultimate measure of success is MFU.

$$MFU = \frac{\text{Observed TFLOPS}}{\text{Theoretical Peak TFLOPS}}$$

- **Target:** A standard PyTorch implementation might hit 30-40%. A highly optimized FlashAttention-3 implementation should target 60-75% on H100.
- **Significance:** This translates directly to massive cost savings at scale, improving operational margins dramatically by extracting greater throughput from existing hardware.

8 The C++ Frontier: High-Performance Serving with PJRT

While Python is the language of research, C++ is the language of production infrastructure. High-performance production systems require integrating Python kernels into the robust C++ serving stacks that power enterprise clouds.

This project phase involves building "Omni-Serve," a lightweight C++ inference runtime.

8.1 The C++ Serving Challenge

In a production environment (like serving a trillion-parameter model), the overhead of the Python Global Interpreter Lock (GIL) is unacceptable for high-throughput request scheduling.

- **The Solution:** We bypass Python entirely for the runtime execution. We export the Pallas/Triton kernels as StableHLO (a portable IR) and load them into a C++ binary using the PJRT (Pre-JIT Runtime) API.

8.2 Implementing the PJRT Client in C++

PJRT is the unified API used to control accelerators (TPU, GPU, CPU) from C++.

- **Implementation:** The C++ code initializes the PJRT client, loads the compiled StableHLO module, and manages the device memory directly.
- **Why this matters:** It demonstrates a deep understanding of how XLA operates natively at the C++ layer, beyond the standard Python frontend.

```cpp
// Conceptual C++ Infrastructure Code (Omni-Serve)
#include "xla/pjrt/c/pjrt_c_api.h"
#include "xla/client/xla_builder.h"

class OmniServeBackend {
public:
    // Initialize the TPU/GPU device client
    void Initialize(std::string device_type) {
        if (device_type == "TPU") {
            client_ = xla::GetTpuClient();
        } else {
            client_ = xla::GetGpuClient();
        }
    }

    // Load the StableHLO (compiled from Pallas)
    void LoadModel(const std::string& hlo_proto_path) {
        std::string serialized_hlo = ReadFile(hlo_proto_path);
        auto executable = client_->Compile(serialized_hlo);
        executables_.push_back(std::move(executable));
    }

    // High-Performance Request Handler
    // No Python overhead here. Direct C++ to Device.
    std::vector<std::shared_ptr<PjRtBuffer>> Execute(const std::vector<void*>&
    input_data) {
        // 1. Move data from Host (CPU) to Device (HBM) using PJRT
        auto device_buffers = client_->BufferFromHost(input_data);

        // 2. Execute the XLA/Pallas Kernel
        // This triggers the optimized attention kernel we wrote in Section 4/5
        auto result = executables_[0]->Execute({device_buffers});
        return result;
    }

private:
    std::shared_ptr<xla::PjRtClient> client_;
    std::vector<std::unique_ptr<xla::PjRtLoadedExecutable>> executables_;
};
```

8.3 Custom C++ Ops: The "Continuous Batching" Scheduler

The project also includes a C++ implementation of a Continuous Batching scheduler.

- **The Problem:** In LLM serving, requests arrive at different times. Waiting for a whole batch to finish is inefficient.
- **C++ Infrastructure:** We implement a `RequestScheduler` class in C++ that manages a Block Table (mapping logical tokens to physical memory blocks).

- **Key Skill:** This requires manual memory management and concurrency (using `std::mutex` and `std::thread` to handle incoming RPCs), addressing critical large-scale infrastructure and software design challenges.

8.4 XLA Custom Calls (XLA_CUSTOM_CALL)

To fully augment the XLA graph, the project includes one Custom Call.

- **Use Case:** A specialized Token Sampling operation (e.g., Top-P with specific constraints) that is inefficient to express in pure HLO.
- **Mechanism:**
  1. Write a C++ function `void CustomTopP(...)`.
  2. Register it with XLA using `cpu_custom_call_target`.
  3. Invoke it from the Pallas/JAX graph.
- **Impact:** This allows extending the compiler's capabilities when the standard library falls short.

9 Strategic Project Structure & Implementation Plan

To serve as a production-grade framework, the code must be structured logically. The proposed repository `omni-attention` is organized as follows:

```
omni_attention/
  layers/
    attention.py          # High-level JAX/Flax Module
    ring_dist.py          # shard_map distributed logic
  kernels/
    triton/
      flash_attn_v3.py    # Hopper-optimized Kernel
      fp8_quant.py        # Block-wise quantization
    pallas/
      tpu_flash.py        # v5e Kernel with BlockSpecs
      tpu_layout.py       # Layout transformation logic
  serving/                # NEW: C++ Infrastructure Layer
    src/
      pjrt_client.cc      # C++ Runtime for XLA
      scheduler.cc        # Continuous Batching Logic
      custom_ops.cc       # XLA Custom Call implementations
    include/
      omni_serve.h
  profiling/
    roofline_plot.py      # Roofline analysis scripts
    benchmark_mfu.py      # MFU calculation
  tests/
    test_numerics.py      # FP8 vs FP32 validation
    test_equivalence.py   # GPU vs TPU output parity
```

9.1 Key Implementation Details: The Pallas Wrapper

Below is the conceptual structure for the Pallas integration, demonstrating the `BlockSpec` and `Ref` usage that defines the TPU implementation.

```python
# Conceptual Pallas Kernel Implementation for TPU v5e
import jax
from jax.experimental import pallas as pl
from jax.experimental.pallas import tpu as pltpu

```

```python
def tpu_flash_attention_kernel(q_ref, k_ref, v_ref, o_ref, smem_ref):
    """Pallas Kernel running on the TPU Core."""
    # q_ref, k_ref, v_ref: Refs to VMEM (Vector Memory)
    # smem_ref: Ref to SMEM (Scalar Memory)

    # 1. Load data from VMEM refs to registers (Implicit in Pallas operations)

    # 2. Compute Dot Product (MXU)
    # The compiler maps this to the matrix unit
    # accumulator in registers
    acc = jnp.zeros(...)

    # 3. Softmax Logic (VPU)
    # Explicitly managed to avoid stalling the MXU
    # Using the smem_ref for efficient scalar reductions
    row_max = jnp.max(acc, axis=-1)
    exp_val = jnp.exp(acc - row_max)

    # 4. Write back
    o_ref[...] = exp_val

# The Call (Grid & BlockSpec)
def call_pallas_flash(q, k, v):
    # Tune these block sizes for TPU v5e VMEM capacity
    B_SEQ, B_HEAD = 128, 128

    # Define the DMA schedule
    grid_spec = pltpu.PrefetchScalarGridSpec(
        num_scalar_prefetch=0,
        in_specs=[...],
        out_specs=pl.BlockSpec((B_SEQ, B_HEAD), lambda i, j: (i, j))
    )

    return pl.pallas_call(
        tpu_flash_attention_kernel,
        out_shape=q,
        grid_spec=grid_spec,
        # Explicitly enabling pipelining (Double Buffering)
        compiler_params=pltpu.TPUCompilerParams(num_pipeline_stages=2)
    )(q, k, v)
```

Analysis of Code: The `BlockSpec` lambda functions define the memory access pattern. The `num_pipeline_stages=2` parameter is the critical optimization. It tells the TPU's DMA engine to fetch the next block of (q, k, v) while the kernel computes the current block. This is the TPU equivalent of the Hopper TMA optimization.

10 Conclusion: A Unified Infrastructure Vision

The "Omni-Attention" project is a strategic demonstration of the advanced techniques required to optimize heterogeneous AI hardware.

1. **TPU Architecture Mastery:** It proves mastery of the TPU stack beyond standard compilation. By using Pallas to manage VMEM and implementing Ring Attention on the ICI torus, it natively optimizes for custom topology interconnects. Furthermore, the Omni-Serve C++ runtime proves the capability to build the production systems that wrap these low-level kernels.

2. **GPU Architecture Mastery:** It displays high competency on the GPU stack beyond CUDA. By using Triton to orchestrate TMA and WGMMA instructions on the H100, and by implementing FP8 block scaling, it directly addresses latency bottlenecks and cost-per-token metrics.

3. **Rigorous Profiling and Distributed Design:** The rigorous use of Roofline analysis, MFU calculation, and `shard_map` distributed overlap serves as irrefutable evidence of robust debugging and scalability at massive scales.

By building Omni-Attention, one transitions from a consumer of standard libraries to an architect of high-performance systems, enabling the next generation of foundation models.

Appendix: Referenced Research Material

The analysis and technical details in this report are derived from the following primary sources:

- **Google TPU & XLA Architecture:** [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]
- **NVIDIA H100 & Triton:** [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]
- **Distributed Systems (Ring Attention/shard_map):** [22], [23], [24], [25], [26]
- **FP8 & Precision:** [27], [28], [29], [30], [31]
- **Profiling & Methodology:** [32], [33], [34], [35]

Works Cited

References

[1] Building production AI on Google Cloud TPUs with JAX, accessed January 27, 2026. https://developers.googleblog.com/building-production-ai-on-google-cloud-tpus-with-jax/

[2] Inside the Ironwood TPU codesigned AI stack — Google Cloud Blog, accessed January 27, 2026. https://cloud.google.com/blog/products/compute/inside-the-ironwood-tpu-codesigned-ai-stack

[3] Google TPU v6e vs GPU: 4x Better AI Performance Per Dollar Guide - Introl, accessed January 27, 2026. https://introl.com/blog/google-tpu-v6e-vs-gpu-4x-better-ai-performance-per-dollar-guide

[4] JAX, XLA, and Pallas - MaxText documentation, accessed January 27, 2026. https://maxtext.readthedocs.io/en/latest/reference/core_concepts/jax_xla_and_pallas.html

[5] Unlocking TPU Performance with Custom MatMul Kernels using JAX Pallas, accessed January 27, 2026. https://sqtian.com/post/pallas_tpu_matmul_kernel/

[6] Pallas Quickstart - JAX documentation, accessed January 27, 2026. https://docs.jax.dev/en/latest/pallas/quickstart.html

[7] The Rise of Pallas: Unlocking TPU Potential with Custom Kernels - Medium, accessed January 27, 2026. https://medium.com/data-science/the-rise-of-pallas-unlocking-tpu-potential-with-custom-kernels-67be10ab846a

[8] Building production AI on Cloud TPUs with JAX, accessed January 27, 2026. https://docs.cloud.google.com/tpu/docs/jax-ai-stack

[9] Using Cloud TPU Multislice to scale AI workloads — Google Cloud Blog, accessed January 27, 2026. https://cloud.google.com/blog/products/compute/using-cloud-tpu-multislice-to-scale-ai-workloads

[10] Using JAX to accelerate our research - Google DeepMind, accessed January 27, 2026. https://deepmind.google/blog/using-jax-to-accelerate-our-research/

[11] FlashAttention-3: Fast and Accurate Attention with Asynchrony and..., accessed January 27, 2026. https://tridao.me/blog/2024/flash3/

[12] FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision - PyTorch, accessed January 27, 2026. https://pytorch.org/blog/flashattention-3/

[13] FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision - Tri Dao, accessed January 27, 2026. https://tridao.me/publications/flash3/flash3.pdf

[14] Flash Attention 3 → Triton Issue #4308 - GitHub, accessed January 27, 2026. https://github.com/triton-lang/triton/issues/4308

[15] What about Triton and Python eDSLs? (Democratizing AI Compute, Part 7) - Modular, accessed January 27, 2026. https://www.modular.com/blog/democratizing-ai-compute-part-7-what-about-triton-and-python-edsls

[16] Kernel development and optimization with Triton - AMD ROCm documentation, accessed January 27, 2026. https://rocm.docs.amd.com/projects/ai-developer-hub/en/v7.0/notebooks/gpu_dev_optimize/triton_kernel_dev.html

[17] OpenAI Triton on NVIDIA Blackwell Boosts AI Performance and Programmability, accessed January 27, 2026. https://developer.nvidia.com/blog/openai-triton-on-nvidia-blackwell-boosts-ai-performance-and-programmability/

[18] FlashAttention: Implementing High-Performance Attention with CUDA and Triton - Medium, accessed January 27, 2026. https://medium.com/@kimdoil1211/flashattention-implementing-high-performance-attention-with-cuda-and-triton-9ee635ab1200

[19] Matrix Multiplication - Triton documentation, accessed January 27, 2026. https://triton-lang.org/main/getting-started/tutorials/03-matrix-multiplication.html

[20] Implementing a Split-K Matrix Multiplication Kernel in Triton — by Michael Diggin — Medium, accessed January 27, 2026. https://medium.com/@michael.diggin/implementing-a-split-k-matrix-multiplication-kernel-in-triton-7ad93fe4a54c

[21] Block Scaled Matrix Multiplication - Triton documentation, accessed January 27, 2026. https://triton-lang.org/main/getting-started/tutorials/10-block-scaled-matmul.html

[22] Topology-aware Decoding for Long-Context Attention on GPU clusters - arXiv, accessed January 27, 2026. https://arxiv.org/html/2408.04093v1

[23] Introduction to parallel programming - JAX documentation, accessed January 27, 2026. https://docs.jax.dev/en/latest/sharded-computation.html

[24] SPMD Parallelism with jax.shard_map and the Evolution Beyond pmap — CodeSignal Learn, accessed January 27, 2026. https://codesignal.com/learn/courses/advanced-jax-transformations-for-speed-scale/lessons/parallel-universes-spmd-parallelism-with-jaxshardmap-and-the-evolution-beyond-pmap

[25] Distributed Computing in Pallas for TPUs - JAX documentation, accessed January 27, 2026. https://docs.jax.dev/en/latest/pallas/tpu/distributed.html

[26] Ring Attention with Blockwise Transformers for Near-Infinite Context - arXiv, accessed January 27, 2026. https://arxiv.org/html/2310.01889v1

[27] User Guide on Using FP8 - Flax Linen, accessed January 27, 2026. https://flax-linen.readthedocs.io/en/v0.11.1/guides/quantization/fp8_basics.html

[28] Precision support - AMD ROCm documentation, accessed January 27, 2026. https://rocm.docs.amd.com/en/docs-6.2.0/compatibility/precision-support.html

[29] not much happened today — AI News - Smol AI, accessed January 27, 2026. https://news.smol.ai/issues/25-11-05-not-much/

[30] e4m3fn vs e5m2 fp8, what one should i use?: r/comfyui - Reddit, accessed January 27, 2026. https://www.reddit.com/r/comfyui/comments/1phcw3n/e4m3fn_vs_e5m2_fp8_what_one_should_i_use/

[31] triton.language.dot_scaled, accessed January 27, 2026. https://triton-lang.org/main/python-api/generated/triton.language.dot_scaled.html

[32] How to Profile TPU Programs — How To Scale Your Model - GitHub Pages, accessed January 27, 2026. https://jax-ml.github.io/scaling-book/profiling/

[33] FlashAttention-3 Baseline Benchmarks - Emergent Mind, accessed January 27, 2026. https://www.emergentmind.com/topics/flashattention-3-baseline

[34] Supercharge ML performance on xPUs with the new XProf profiler and Cloud Diagnostics XProf library — Google Cloud Blog, accessed January 27, 2026. https://cloud.google.com/blog/topics/developers-practitioners/supercharge-ml-performance-on-xpus-with-the-new-xprof-profiler-and-cloud-diagnostics-xpro

[35] Profiling computation - JAX documentation, accessed January 27, 2026. https://docs.jax.dev/en/latest/profiling.html